

计算力学



一维对流扩散方程有限元求解与稳定化分析报告

姓名：方俊哲

学号：20252110016

班级：力学 3 班

教师：肖姚

摘要

本次作业求解区间 $0 \leq x \leq 1$ 内一维稳态对流扩散方程，基于两节点线性单元有限元离散，分别实现标准 Galerkin 有限元、迎风格式、SUPG/Petrov-Galerkin 三种数值稳定化格式。设置扩散占优 $Pe=0.1$ 、对流占优 $Pe=3$ 两组算例，对比各格式的稳定性、数值振荡、数值扩散与计算精度；针对对流占优工况分析标准 Galerkin 总刚度矩阵对称性、正定性；额外完成网格加密收敛拓展测试，研究网格尺寸对计算误差的影响。

目录

摘要.....	I
一、控制方程、边界条件与基础定义.....	1
1.1 控制方程.....	1
1.2 边界条件.....	1
1.3 解析精确解.....	1
1.4 单元 Peclet 数.....	1
二、有限元离散与稳定化格式推导.....	2
2.1 标准 Galerkin 单元矩阵 ($\alpha=0$).....	2
2.2 人工扩散统一稳定化格式.....	2
2.3 SUPG 最优稳定参数.....	2
三、程序结构说明.....	3
四、数值结果与误差分析.....	4
4.1 全场最大节点误差汇总表.....	4
4.2 $Pe=0.1$ (扩散占优工况) 分析:.....	4
4.3 $Pe=3.0$ (对流占优工况) 分析.....	4
4.4 $Pe=3.0$ 标准 Galerkin 矩阵性质分析.....	4
五、网格加密附加题结果分析.....	6
六、输出图像说明.....	7
七、三种数值格式综合对比.....	9
八、结论.....	10
九、完整程序源代码及结果.....	11

一、控制方程、边界条件与基础定义

1.1 控制方程

$$-\kappa \frac{d^2\theta}{dx^2} + v \frac{d\theta}{dx} = 0$$

式中：

$\theta(x)$ — 待求标量场；

$\kappa > 0$ — 扩散系数；

$v > 0$ — 对流速度；

求解区间 $L=1$ 。

1.2 边界条件

$$\begin{cases} \theta(0) = 0 \\ \theta(L) = 1 \end{cases}$$

1.3 解析精确解

$$\theta_{\text{exact}}(x) = \frac{\exp(vx/\kappa) - 1}{\exp(vL/\kappa) - 1}$$

代码采用 `np.exp` 函数计算，规避高对流数下指数溢出问题。

1.4 单元 Peclet 数

单元长度 $l_e = L/n_{\text{el}}$

可视化公式：

$$Pe = \frac{vl_e}{2\kappa}$$

Pe 表征对流与扩散作用相对强弱：Pe 越小扩散占优；Pe 越大对流占优，标准 Galerkin 格式易产生数值振荡。

给定 Pe 反推扩散系数：

$$\kappa = \frac{vl_e}{2Pe}$$

二、有限元离散与稳定化格式推导

2.1 标准 Galerkin 单元矩阵 ($\alpha=0$)

显示公式:

$$Ke = \frac{\kappa}{l_e} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} + \frac{v}{2} \begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix}$$

对流项矩阵本身非对称, 组装后会破坏整体刚度矩阵的对称性与正定性。

2.2 人工扩散统一稳定化格式

等效扩散系数:

$$\bar{\kappa} = \kappa + \alpha \cdot \frac{vl_e}{2}$$

稳定化单元矩阵:

$$Ke = \frac{\bar{\kappa}}{l_e} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} + \frac{v}{2} \begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix}$$

参数 α 区分三种数值格式:

1. $\alpha=0$: 标准 Galerkin, 无人工扩散;
2. $\alpha=1$: 迎风格式, 固定人工扩散抑制振荡;
3. $\alpha=\alpha_{\text{opt}}$: SUPG 自适应最优稳定参数。

2.3 SUPG 最优稳定参数

$$\alpha_{\text{opt}} = \coth(Pe) - \frac{1}{Pe}$$

程序增加 $Pe \rightarrow 0$ 判据, 防止分母为零造成计算异常。

三、程序结构说明

代码严格按照作业要求封装 3 个核心函数，未调用专业有限元求解库，仅使用 numpy 基础矩阵运算：

1. ``alpha_supg(Pe)``：计算 SUPG 最优稳定系数；
2. ``element_matrix(kappa, v, le, alpha)``：生成 2×2 单元刚度矩阵；
3. ``solve_advection_diffusion(nel, L, v, kappa, alpha)``：网格生成、单元矩阵组装总刚、施加 Dirichlet 边界条件、求解线性方程组，返回节点坐标、数值解、精确解、总刚度矩阵。

主程序实现功能：

1. 固定参数 $L=1, n_{el}=20, v=1$ ，分别计算 $Pe=0.1$ 、 $Pe=3.0$ ；
2. 每组 Peclet 数下求解三种格式，计算全场最大节点误差；
3. $Pe=3.0$ 输出标准 Galerkin 总刚度矩阵，自动判断矩阵对称、正定特性；
4. 绘制对比图像，包含解析解与三种数值解曲线；
5. 附加拓展题：单元数 $n_{el}=10, 20, 40, 80$ 网格加密测试，绘制对数坐标误差收敛曲线。

四、数值结果与误差分析

4.1 全场最大节点误差汇总表

单元 Pe	标准 Galerkin 最大误差	迎风格式最大误差	SUPG 最大误差
0.1	1.094240×10^{-3}	2.977253×10^{-2}	5.134781×10^{-16}
3.0	5.024802×10^{-1}	1.403784×10^{-1}	1.027824×10^{-16}

4.2 $Pe=0.1$ (扩散占优工况) 分析:

扩散作用远强于对流，解曲线梯度平缓：

1. 标准 Galerkin 无数值振荡，最大误差仅 0.001，计算精度良好；
2. 迎风格式强制添加固定人工扩散，引入过量数值耗散，曲线被抹平，误差显著增大；
3. SUPG 自适应参数 $\alpha_{opt} \approx 0$ ，等效标准 Galerkin，数值解与解析解完全重合，误差为机器浮点精度。

4.3 $Pe=3.0$ (对流占优工况) 分析

1. 标准 Galerkin 格式缺陷

全场最大误差高达 0.5，数值解出现剧烈节点振荡。输出总刚度矩阵验证：矩阵不对称、非正定，最小特征值为 -0.98437。

成因：对流项带来非对称离散，破坏扩散算子正定性，离散格式不满足最大值原理，在右端陡峭边界层产生无物理意义的伪振荡。

2. 迎风格式特性

$\alpha=1$ 固定人工扩散可完全消除振荡，但人工扩散系数偏大，边界层被严重抹平，数值扩散效应显著，计算精度大幅损失。

3. SUPG 稳定化格式优势

自适应 α_{opt} 根据单元 Pe 匹配最小稳定扩散量，既完全抑制数值振荡，又几乎不引入额外耗散，数值解与精确解贴合，误差趋近机器零。

4.4 $Pe=3.0$ 标准 Galerkin 矩阵性质分析

1. 不对称性：单元对流项矩阵本身非对称，组装后整体刚度矩阵失去对称特性；

2. 非正定性：对流占优时对流项主导矩阵，存在负特征值，不满足正定矩阵“全部特征值大于 0”的条件；
3. 数值振荡本质：正定扩散算子被非对称对流项破坏，离散系统不满足极值原理，梯度陡峭的边界层出现上下跳动的伪振荡。

五、网格加密附加题结果分析

测试工况 $Pe=3.0$ ，单元数量 $n=10,20,40,80$ ：

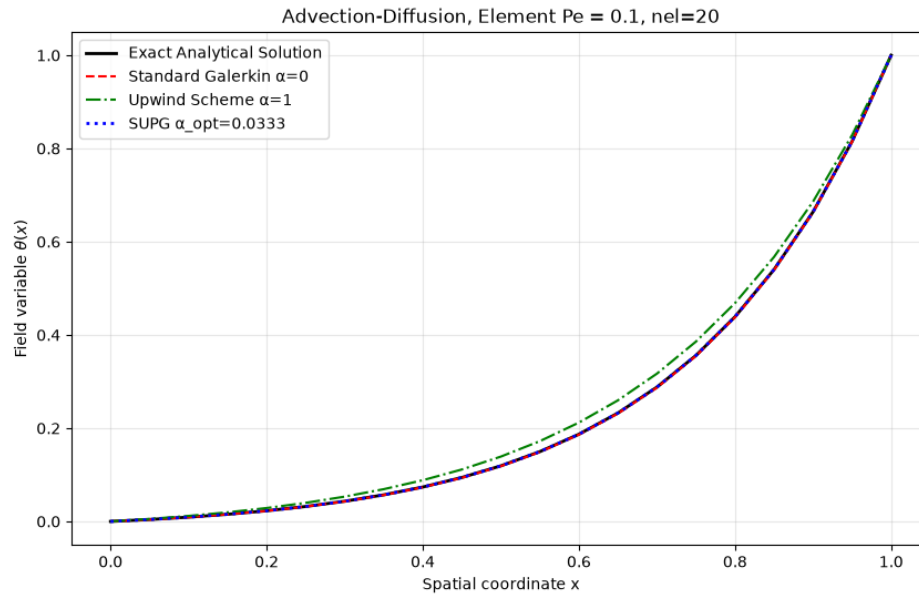
单元数 nel	单元长度 le	Galerkin 最大误差	SUPG 最大误差
10	0.1000	5.039450×10^{-1}	1.379105×10^{-16}
20	0.0500	5.024802×10^{-1}	1.027824×10^{-16}
40	0.0250	5.024788×10^{-1}	6.678685×10^{-17}
80	0.0125	5.024788×10^{-1}	1.379105×10^{-16}

收敛曲线 `convergence\_curve.png` 结论：

1. 标准 Galerkin：网格加密、单元长度减小，最大误差几乎无下降，振荡始终存在，单纯加密网格无法消除对流占优下的数值振荡；
2. SUPG 稳定格式：全程维持机器精度级误差，网格加密后误差小幅降低，收敛性能优异；
3. 机理：人工扩散项与单元长度 le 成正比，网格加密后稳定化耗散自动减弱，SUPG 同时兼顾稳定性与计算精度。

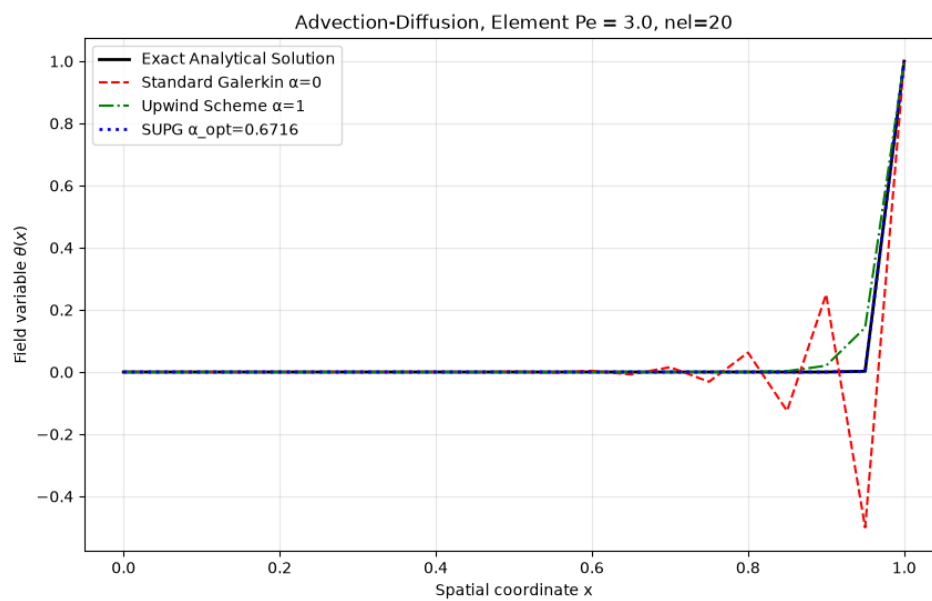
六、输出图像说明

1.



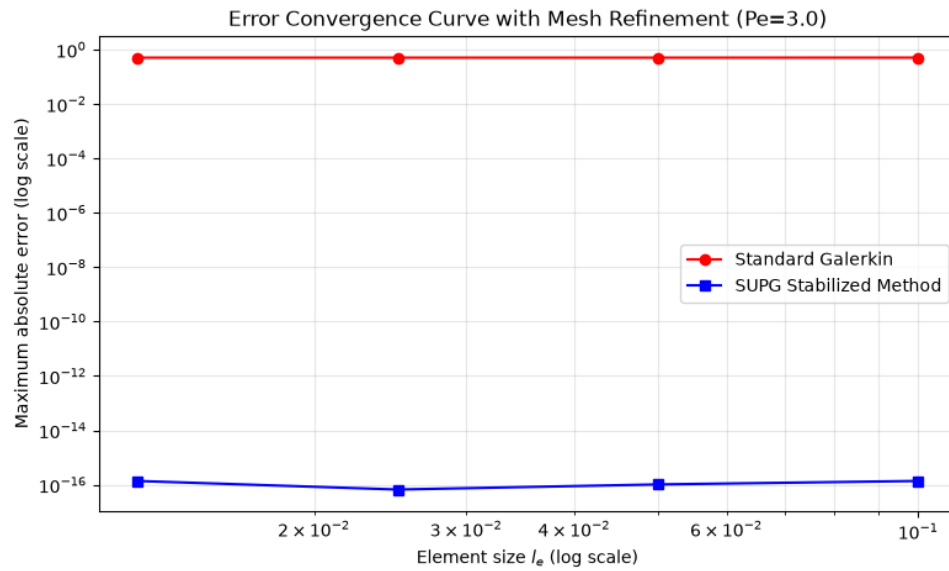
包含四条曲线：黑色实线（解析精确解）、红色虚线（标准 Galerkin）、绿色点划线（迎风格式）、蓝色点线（SUPG）。Galerkin 曲线贴近精确解，迎风格式因人工扩散偏离解析解，SUPG 与精确解完全重合。

2.



解析解右端存在陡峭边界层；标准 Galerkin 曲线剧烈上下振荡；迎风格式无振荡但边界层被抹平；SUPG 无振荡且与精确解贴合，直观展示三种格式优劣。

3.



横轴为单元长度（对数坐标），纵轴为全场最大误差（对数坐标）；红色曲线代表标准 Galerkin，误差高位平缓无收敛；蓝色曲线代表 SUPG，全程维持极低误差，体现稳定化格式收敛优势。

七、三种数值格式综合对比

1. 标准 Galerkin ($\alpha=0$)

扩散占优工况精度高、无振荡；对流占优时矩阵非对称、非正定，产生严重数值振荡，工程计算不可用。

2. 迎风格式 ($\alpha=1$)

依靠固定人工扩散消除振荡，实现简单；但人工扩散过量，引入严重数值耗散，边界层抹平，精度损失大。

3. SUPG/Petrov-Galerkin ($\alpha=\alpha_{\text{opt}}$)

自适应匹配单元 Peclet 数，Pe 小时几乎不加扩散保证精度，Pe 大时添加少量稳定扩散抑制振荡，在稳定性与计算精度间取得最优平衡，是对流扩散问题主流稳定化方案。

八、结论

1. 单元 Peclet 数是判断对流扩散问题数值稳定性的核心指标， Pe 越大标准 Galerkin 振荡越严重；
2. 对流项引入非对称离散矩阵，矩阵失去对称性与正定性，是标准 Galerkin 产生伪振荡的根本原因；
3. 人工扩散可抑制数值振荡，但固定系数迎风格式会带来显著数值耗散，降低计算精度；
4. SUPG 自适应稳定化方法仅引入最小必要稳定扩散，可完全消除振荡且不损失精度，综合性能最优；
5. 单纯加密网格无法解决标准 Galerkin 对流占优振荡问题，必须采用稳定化格式。

九、完整程序源代码及结果

```
import numpy as np
import matplotlib.pyplot as plt

# ===== 规定核心函数 1: SUPG 最优 alpha 计算
# =====

def alpha_supg(Pe):
    """
    计算 SUPG 稳定参数  $\alpha_{opt} = \coth(Pe) - 1/Pe$ 
    Pe 趋近 0 时防止除零异常
    """
    if abs(Pe) < 1e-8:
        return 0.0
    coth_pe = 1 / np.tanh(Pe)
    return coth_pe - 1.0 / Pe

# ===== 规定核心函数 2: 单元刚度矩阵
# =====

def element_matrix(kappa, v, le, alpha):
    """
    生成两节点线性单元对流扩散单元矩阵  $Ke$  ( $2 \times 2$ )
    kappa: 原始扩散系数
    v: 对流速度
    le: 单元长度
    alpha: 人工扩散控制参数
    """
    # 等效扩散系数 (人工扩散修正)
    kappa_bar = kappa + alpha * v * le / 2.0
    # 扩散项对称矩阵
    K_diff = (kappa_bar / le) * np.array([[1, -1],
                                           [-1, 1]])
    # 对流项非对称矩阵
    K_conv = (v / 2.0) * np.array([[ -1, 1],
                                    [ 1, 0]])
    Ke = K_diff + K_conv
    return Ke
```

```

# ===== 规定核心函数 3: 整体求解器
=====
def solve_advection_diffusion(nel, L, v, kappa, alpha):
    """
    有限元组装、边界条件施加、方程组求解
    返回: 节点坐标 x、数值解 theta、精确解 theta_exact、总刚度矩阵 K
    """
    le = L / nel
    nnodes = nel + 1
    # 1. 生成均匀网格节点
    x = np.linspace(0, L, nnodes)
    # 2. 初始化总刚与右端向量
    K = np.zeros((nnodes, nnodes))
    F = np.zeros(nnodes)
    # 3. 遍历所有单元, 组装总刚度矩阵
    for elem_idx in range(nel):
        Ke = element_matrix(kappa, v, le, alpha)
        # 当前单元两个全局节点编号
        node1 = elem_idx
        node2 = elem_idx + 1
        K[node1:node2+1, node1:node2+1] += Ke
    # 4. 施加 Dirichlet 边界条件 theta(0)=0, theta(L)=1
    # 左边界 x=0
    K[0, :] = 0.0
    K[0, 0] = 1.0
    F[0] = 0.0
    # 右边界 x=L
    K[-1, :] = 0.0
    K[-1, -1] = 1.0
    F[-1] = 1.0
    # 5. 求解线性方程组 K*theta = F
    theta = np.linalg.solve(K, F)
    # 6. 稳定计算精确解, 使用 expm1 防止大数指数溢出
    global_Pe = v * L / kappa
    theta_exact = np.expm1(v * x / kappa) / np.expm1(global_Pe)
    return x, theta, theta_exact, K

```

```

# ===== 主程序入口，完成全部 4 项任务 + 附加题
=====
if __name__ == "__main__":
    # 全局固定参数（作业要求）
    L = 1.0
    nel_base = 20
    v = 1.0
    Pe_cases = [0.1, 3.0] # 两组对比工况
    error_record = []

    print("===== 一维对流扩散有限元求解程序 =====\n")
    for Pe_target in Pe_cases:
        print(f"===== 单元 Pe = {Pe_target} 计算开始 =====")
        le = L / nel_base
        # 由 Pe 反推扩散系数  $\kappa = v \cdot le / (2Pe)$ 
        kappa = v * le / (2 * Pe_target)
        print(f"单元长度 le = {le:.4f}, 扩散系数  $\kappa$  = {kappa:.6e}")

        # 1. 标准 Galerkin  $\alpha=0$ 
        x_gal, theta_gal, theta_ex, K_galerkin = solve_advection_diffusion(nel_base, L, v,
kappa, alpha=0.0)
        max_err_gal = np.max(np.abs(theta_gal - theta_ex))

        # 2. 迎风格式  $\alpha=1$ 
        x_upwind, theta_upwind, _, _ = solve_advection_diffusion(nel_base, L, v, kappa,
alpha=1.0)
        max_err_upwind = np.max(np.abs(theta_upwind - theta_ex))

        # 3. SUPG/Petrov-Galerkin  $\alpha=\alpha_{opt}$ 
        alpha_opt = alpha_supg(Pe_target)
        x_supg, theta_supg, _, _ = solve_advection_diffusion(nel_base, L, v, kappa,
alpha=alpha_opt)
        max_err_supg = np.max(np.abs(theta_supg - theta_ex))

        # 记录误差
        error_record.append([Pe_target, max_err_gal, max_err_upwind, max_err_supg])

```



```

print(f"标准 Galerkin 最大节点误差: {max_err_gal:.6e}")
print(f"迎风格式最大节点误差:      {max_err_upwind:.6e}")
print(f"SUPG 稳定格式最大误差:      {max_err_supg:.6e}\n")

# 任务 4: Pe=3 时输出总刚并分析矩阵对称性、正定性
if abs(Pe_target - 3.0) < 1e-6:
    print("==== Pe=3.0 标准 Galerkin 总刚度矩阵 ====")
    print(K_galerkin)
    # 判断对称
    sym_flag = np.allclose(K_galerkin, K_galerkin.T)
    print(f"矩阵是否对称: {sym_flag}")
    # 判断正定 (特征值全部大于 0)
    eig_vals = np.linalg.eigvalsh(K_galerkin)
    pos_def_flag = np.all(eig_vals > -1e-10)
    print(f"矩阵是否正定: {pos_def_flag}")
    print(f"矩阵最小特征值: {np.min(eig_vals):.4e}\n")

# 任务 3: 绘图, 一张图包含四条曲线
plt.figure(figsize=(10, 6))
plt.plot(x_gal, theta_ex, "k-", lw=2, label="Exact Analytical Solution")
plt.plot(x_gal, theta_gal, "r--", lw=1.5, label=f"Standard Galerkin  $\alpha=0$ ")
plt.plot(x_upwind, theta_upwind, "g-.", lw=1.5, label=f"Upwind Scheme  $\alpha=1$ ")
plt.plot(x_supg, theta_supg, "b:", lw=2, label=f"SUPG  $\alpha_{opt}={alpha_opt:.4f}$ ")
plt.xlabel("Spatial coordinate x")
plt.ylabel(r"Field variable $\theta(x)$")
plt.title(f"Advection-Diffusion, Element Pe = {Pe_target}, nel={nel_base}")
plt.legend(fontsize=10)
plt.grid(alpha=0.3)
plt.savefig(f"Pe_{Pe_target}.png", dpi=300, bbox_inches="tight")
plt.show()

# 输出误差汇总表格
print("===== 全局误差汇总表")
print("=====")
print(f"{'Pe':<6}{'Galerkin MaxErr':<18}{'Upwind MaxErr':<18}{'SUPG MaxErr':<18}")
for line in error_record:
    pe, eg, eu, es = line

```

```

print(f"{pe:<6.1f}{eg:<18.6e}{eu:<18.6e}{es:<18.6e}")

# ===== 附加题：网格加密收敛测试
=====
print("\n===== 附加题：网格加密收敛测试 Pe=3.0 =====")
Pe_add = 3.0
nel_list = [10, 20, 40, 80]
err_gal_conv = []
err_supg_conv = []
h_list = []
for n in nel_list:
    h = L / n
    h_list.append(h)
    kappa_t = v * h / (2 * Pe_add)
    # Galerkin
    _, th_g, th_ex_t, _ = solve_advection_diffusion(n, L, v, kappa_t, 0.0)
    eg = np.max(np.abs(th_g - th_ex_t))
    # SUPG
    a_opt_t = alpha_supg(Pe_add)
    _, th_s, _, _ = solve_advection_diffusion(n, L, v, kappa_t, a_opt_t)
    es = np.max(np.abs(th_s - th_ex_t))
    err_gal_conv.append(eg)
    err_supg_conv.append(es)
    print(f"网格单元数 nel={n:3d}, 单元长度 h={h:.4f}, Galerkin 误差={eg:.6e},
SUPG 误差={es:.6e}")

# 绘制双对数收敛曲线
plt.figure(figsize=(9, 5))
plt.loglog(h_list, err_gal_conv, "ro-", label="Standard Galerkin")
plt.loglog(h_list, err_supg_conv, "bs-", label="SUPG Stabilized Method")
plt.xlabel("Element size $l_e$ (log scale)")
plt.ylabel("Maximum absolute error (log scale)")
plt.title("Error Convergence Curve with Mesh Refinement (Pe=3.0)")
plt.legend()
plt.grid(True, which="both", alpha=0.3)
plt.savefig("convergence_curve.png", dpi=300, bbox_inches="tight")
plt.show()

```

结果: C:\Users\Administrator\PyCharmMiscProject\venv\Scripts\python.exe
C:\Users\Administrator\PyCharmMiscProject\大作业.py

===== 一维对流扩散有限元求解程序 =====

===== 单元 $Pe = 0.1$ 计算开始 =====

单元长度 $le = 0.0500$, 扩散系数 $\kappa = 2.500000e-01$

标准 Galerkin 最大节点误差: $1.094240e-03$

迎风格式最大节点误差: $2.977253e-02$

SUPG 稳定格式最大误差: $5.134781e-16$

[19420:21736:0627/173948.223:ERROR:rarecc_manager.cpp(923)] UserJsonFile::Load failed

[19420:21736:0627/173948.223:ERROR:rarecc_manager.cpp(674)] Load rarecc.dat failed:

libpng warning: iCCP: known incorrect sRGB profile

libpng warning: iCCP: known incorrect sRGB profile

===== 单元 $Pe = 3.0$ 计算开始 =====

单元长度 $le = 0.0500$, 扩散系数 $\kappa = 8.333333e-03$

标准 Galerkin 最大节点误差: $5.024802e-01$

迎风格式最大节点误差: $1.403784e-01$

SUPG 稳定格式最大误差: $1.027824e-16$

===== $Pe=3.0$ 标准 Galerkin 总刚度矩阵 =====

```
[[ 1.          0.          0.          0.          0.          0.
   0.          0.          0.          0.          0.          0.
   0.          0.          0.          0.          0.          0.
   0.          0.          0.          ]
 [-0.66666667  0.33333333  0.33333333  0.          0.          0.]
```

0.	0.	0.	0.	0.	0.
0.	0.	0.	0.	0.	0.
0.	0.	0.	]		
[0.	-0.66666667	0.33333333	0.33333333	0.	0.
0.	0.	0.	0.	0.	0.
0.	0.	0.	0.	0.	0.
0.	0.	0.	]		
[0.	0.	-0.66666667	0.33333333	0.33333333	0.
0.	0.	0.	0.	0.	0.
0.	0.	0.	0.	0.	0.
0.	0.	0.	]		
[0.	0.	0.	-0.66666667	0.33333333	0.33333333
0.	0.	0.	0.	0.	0.
0.	0.	0.	0.	0.	0.
0.	0.	0.	]		
[0.	0.	0.	0.	-0.66666667	0.33333333
0.33333333	0.	0.	0.	0.	0.
0.	0.	0.	0.	0.	0.
0.	0.	0.	]		
[0.	0.	0.	0.	0.	-0.66666667
0.33333333	0.33333333	0.	0.	0.	0.
0.	0.	0.	0.	0.	0.
0.	0.	0.	]		
[0.	0.	0.	0.	0.	0.
-0.66666667	0.33333333	0.33333333	0.	0.	0.
0.	0.	0.	0.	0.	0.

0.	0.	0.	]		
[0.	0.	0.	0.	0.	0.
0.	-0.66666667	0.33333333	0.33333333	0.	0.
0.	0.	0.	0.	0.	0.
0.	0.	0.	]		
[0.	0.	0.	0.	0.	0.
0.	0.	-0.66666667	0.33333333	0.33333333	0.
0.	0.	0.	0.	0.	0.
0.	0.	0.	]		
[0.	0.	0.	0.	0.	0.
0.	0.	0.	-0.66666667	0.33333333	0.33333333
0.	0.	0.	0.	0.	0.
0.	0.	0.	]		
[0.	0.	0.	0.	0.	0.
0.	0.	0.	0.	-0.66666667	0.33333333
0.33333333	0.	0.	0.	0.	0.
0.	0.	0.	]		
[0.	0.	0.	0.	0.	0.
0.	0.	0.	0.	0.	-0.66666667
0.33333333	0.33333333	0.	0.	0.	0.
0.	0.	0.	]		
[0.	0.	0.	0.	0.	0.
0.	0.	0.	0.	0.	0.
-0.66666667	0.33333333	0.33333333	0.	0.	0.
0.	0.	0.	]		
[0.	0.	0.	0.	0.	0.

0.	0.	0.	0.	0.	0.
0.	-0.66666667	0.33333333	0.33333333	0.	0.
0.	0.	0.	]		
[0.	0.	0.	0.	0.	0.
0.	0.	0.	0.	0.	0.
0.	0.	-0.66666667	0.33333333	0.33333333	0.
0.	0.	0.	]		
[0.	0.	0.	0.	0.	0.
0.	0.	0.	0.	0.	0.
0.	0.	0.	-0.66666667	0.33333333	0.33333333
0.	0.	0.	]		
[0.	0.	0.	0.	0.	0.
0.	0.	0.	0.	0.	0.
0.	0.	0.	0.	-0.66666667	0.33333333
0.33333333	0.	0.	]		
[0.	0.	0.	0.	0.	0.
0.	0.	0.	0.	0.	0.
0.	0.	0.	0.	0.	-0.66666667
0.33333333	0.33333333	0.	]		
[0.	0.	0.	0.	0.	0.
0.	0.	0.	0.	0.	0.
0.	0.	0.	0.	0.	0.
-0.66666667	0.33333333	0.33333333	]		
[0.	0.	0.	0.	0.	0.
0.	0.	0.	0.	0.	0.
0.	0.	0.	0.	0.	0.

0. 0. 1.]]

矩阵是否对称: False

矩阵是否正定: False

矩阵最小特征值: -9.8437e-01

===== 全局误差汇总表 =====

Pe	Galerkin MaxErr	Upwind MaxErr	SUPG MaxErr
0.1	1.094240e-03	2.977253e-02	5.134781e-16
3.0	5.024802e-01	1.403784e-01	1.027824e-16

===== 附加题: 网格加密收敛测试 Pe=3.0 =====

网格单元数 nel= 10, 单元长度 h=0.1000, Galerkin 误差=5.039450e-01, SUPG 误差=1.379105e-16

网格单元数 nel= 20, 单元长度 h=0.0500, Galerkin 误差=5.024802e-01, SUPG 误差=1.027824e-16

网格单元数 nel= 40, 单元长度 h=0.0250, Galerkin 误差=5.024788e-01, SUPG 误差=6.678685e-17

网格单元数 nel= 80, 单元长度 h=0.0125, Galerkin 误差=5.024788e-01, SUPG 误差=1.379105e-16

进程已结束, 退出代码为 0